

Questão 4-

a) Há deadlock clássico? Explique.

Não. Não há deadlock clássico porque nenhuma thread fica presa indefinidamente em uma chamada bloqueante de *pthread_mutex_lock*. Graças ao *pthread_mutex_trylock*, se o recurso estiver ocupado, a thread desiste da espera, libera o que já possui e tenta novamente.

b) Há outro problema concorrente possível? Qual?

Sim, o *Livelock*. Se o escalonador do sistema operacional alternar as threads de forma simétrica, ambas as threads podem adquirir seus respectivos primeiros mutexes (*t1* pega *m1* e *t2* pega *m2*), falhar em adquirir o segundo recurso, liberar o primeiro e repetir esse ciclo infinitamente sem progredir para a área de execução.

c) Quais condições de impasse foram quebradas e quais permanecem?

Quebrada:

A condição de Posse e Espera foi mitigada dinamicamente. A thread não fica "segurando uma e esperando a outra para sempre". Se ela não consegue a segunda, ela renuncia à posse da primeira. A Espera Circular também deixa de ser estática.

Permanecem:

Exclusão Mútua (os *mutexes* ainda são estritamente exclusivos) e Não-Preempção (o SO não toma o *mutex* de uma thread à força).

d) Reescreva o código de maneira mais robusta.

Solução no arquivo *resolucao4.c*